1.Signals from `adj_close` but filled trades at the raw `open`. What exactly is wrong with that, under what conditions does it change the result, and what is the one-line fix in your own code?

→ My directional-change detector (dc_core.py) and the HMM (hmm_strategy.py) both operate purely on candle.prices, which is adj_close. But engine.py fills every trade off candle.opens
→Dividends:- adj_close typically bakes dividends into the adjustment factor; raw open/close do not. On an ex-dividend date the raw price drops by the dividend amount, but the engine's cash accounting never credits that dividend to the holder.
Adj_close ~ close
Open ~adj_open
→Return becomes inconsistent because entry and exit prices are measured on a different scale than the signal generation process.
Example:- entry_signal at adj_close = 100
After 2-for-1 split
Next day, open = 50 , adj_open = 100
Whenever corporate action occurs, it happened.

—>In loader.py One line code change the from

```
@property
def opens(self):
    return self.df["adj_open"].to_numpy(dtype=float)
```

2.Why is buy-and-hold the wrong benchmark for a long/flat strategy on a stock that lost 43% over the test window? What is the right one?

→A long/flat strategy goes to cash during downturns. Comparing it to a continuously bleeding asset penalizes the strategy for missing losses.
Ex:- The stock→x%  Cash—>100-x%
→Volatility and drawdowns for the flat periods are completely different from a static position.
Right Benchmark:-
→Benchmark related to similar to alpha/strategy form(like given Task part2)

→A benchmark that mimics the exact daily exposure limits of active strategy.

→Blends the asset's return only on days the strategy was actually long, and the risk-free rate on days it was flat.

3.A back-adjusted price series is rewritten every time a dividend is paid. What does that mean for the difference between what your backtest saw and what a live system would have seen on each historical day?

→A back-adjusted series is built backwards from the *most recent* price. Whenever a dividend is paid, the data vendor recalculates an adjustment factor and applies it to all previous prices in the series.

F_d = (close_d - dividend_d) / close_d

And multiplies every price before date d by F_d.

AdjustedPrice(t) = RawPrice(t) × AdjustmentFactor(T_future)

→A live system trading has no access to dividends that haven't happened yet. It can only know the raw price, and at best an adjustment factor for dividends paid *up to and including* that day.

→The adjusted price on a historical day depends on corporate actions that occurred in the future. Indicators calculated from the adjusted series (moving averages, momentum, returns, etc.) may differ from the values that a live trading system would have computed on that day.

4.In your 1-minute submission, the up/flat/down label thresholds were quantiles computed over the full sample, including the test period. Why does that matter even though the model never saw test rows?

→The Up/Flat/Down thresholds are estimated from the entire forward-return distribution before the train/test split. As a result, future test-period returns influence the label definitions used during training.

→A more correct approach would be to estimate the quantile thresholds using only the training period and then apply those fixed thresholds to both the training and test sets.

Correct way to do it:- Dataset → train/test split →compute on Train only → create train labels →now test labels

My method:- create labels on full data → train/test split

→if it is applied to the entire dataset before the train/test split, the quantiles are influenced by future test-period returns, introducing look-ahead bias. The thresholds should ideally be estimated using only the training period and then applied unchanged to the test period.